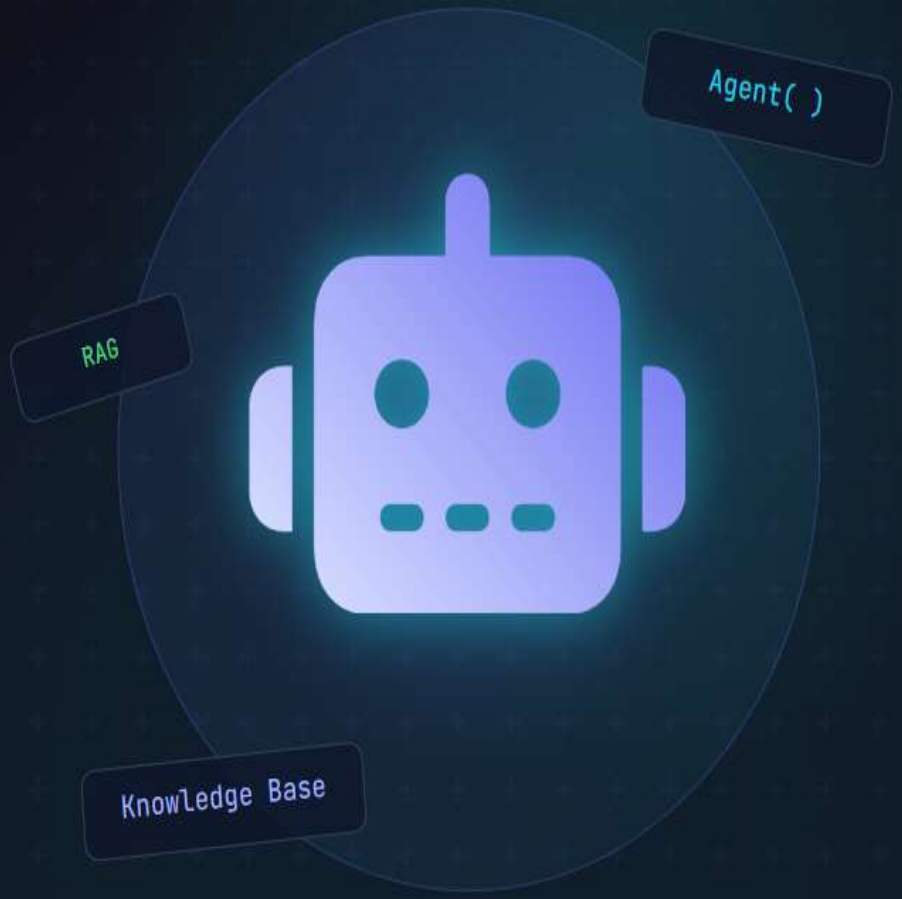
PYTHON

Agno Framework

Создание **агента-помощника** по русскоязычной документации

От прототипа до production API за 20+ строк кода



github.com/agno-agi/agno

docs.agno.com

Что такое Agno?

Agno — это runtime для агентного ПО: создавайте агентов, команды и workflows, запускайте их как масштабируемые сервисы и управляйте в production.

Framework

`agno.agent`

- Создание агентов
- Управление памятью
- Знания (RAG)
- Guardrails
- 100+ интеграций
- Команды агентов

Runtime

`agno.os`

- Stateless FastAPI
- Изоляция сессий
- Стриминг ответов
- Auth & Permissions
- Масштабирование
- Docker container

Control Plane

`AgentOS UI`

- Веб-интерфейс
- Тестирование
- Мониторинг
- Трассировка (Tracing)
- Управление агентами
- Аудит действий

Ключевые возможности Agno

Память

Встроенная поддержка долговременной памяти и истории сессий. Агенты запоминают контекст разговора, предпочтения пользователей и факты между сессиями (SQLite, Postgres).

База знаний

Agentic RAG позволяет агентам самостоятельно искать информацию в векторных базах данных (Qdrant, PgVector) для ответов на вопросы по документации.

Инструменты

Более 100 готовых интеграций и поддержка протокола MCP. Агенты могут использовать API, искать в интернете, работать с файлами и базами данных.

Guardrails

Встроенные механизмы безопасности и валидации. Контролируйте ввод и вывод модели, предотвращайте галлюцинации и утечки данных на уровне framework.

Команды

Создание мульти-агентных систем, где специализированные агенты работают вместе. Координация ролей, передача задач и обмен контекстом между агентами.

Мониторинг

Полная прозрачность работы агента через AgentOS UI. Трассировка (tracing) каждого шага рассуждений, аудит вызовов инструментов и метрики производительности.

Установка и настройка

1 Установка пакетов

```
# Основной пакет + AgentOS
pip install -U "agno[os]"

# Для работы с PDF и Knowledge Base
pip install "agno[pdf]" qdrant-client pgvector
```

2 Экспорт API ключей

```
# macOS / Linux
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."

# Windows PowerShell
$env:OPENAI_API_KEY="sk-..."
```

ПОДДЕРЖИВАЕМЫЕ МОДЕЛИ

OpenAI	Anthropic	Google	Ollama	Groq	Mistral
GPT-4o	Claude 3.5	Gemini 1.5	Local Llama 3	LPU Inference	Large

Структура базового агента

agent.py

```python
from agno.agent import Agent
from agno.models.openai import OpenAIChat

# Инициализация агента
agent = Agent(
    name="RU Docs Assistant",
    model=OpenAIChat(id="gpt-4o"),
    description="Помощник по русскоязычной документации",
    instructions=[
        "Отвечай всегда на русском языке",
        "Используй технический стиль общения",
        "Приводи примеры кода в блоках markdown",
    ],
    markdown=True,
    show_tool_calls=True,
)

# Запуск в CLI (для теста)
agent.print_response("Как создать агента в Agno?")
```

name

Уникальное имя агента для идентификации в логах и UI.

model

LLM-провайдер (OpenAI, Anthropic, Ollama и др.) и конкретная версия модели.

description

Краткое описание роли агента, которое помогает при оркестрации команд.

instructions

Список системных инструкций (System Prompt). Определяют поведение, стиль и правила.

markdown

Форматирование вывода. Если True, ответ будет красиво отформатирован в Markdown.

База знаний: поддерживаемые типы

PDFKnowledgeBase

Загрузка и чтение локальных PDF файлов. Идеально для внутренней документации.

PDFUrlKnowledgeBase

Загрузка PDF документов по прямым ссылкам (URL) из интернета.

TextKnowledgeBase

Работа с текстовыми файлами (.txt) и документами Word (.docx).

WebsiteKB

Скрапинг и индексация веб-страниц для создания базы знаний из сайтов.

CSVKnowledgeBase

Индексация структурированных данных из таблиц CSV для аналитики.

JSONKnowledgeBase

Обработка JSON файлов со сложной структурой данных.

ArXivKB

Поиск и индексация научных статей с портала ArXiv.org.

WikipediaKB

Получение знаний из статей Википедии по заданным темам.

Agentic RAG

В отличие от обычного RAG, в Agno агент **самостоятельно решает**, когда и что искать в базе знаний. Он формирует поисковые запросы на основе контекста диалога и может уточнять поиск, если информации недостаточно.

Agentic RAG: как это работает

КЛЮЧЕВЫЕ ЭТАПЫ РАБОТЫ

1 Индексация

Загрузка документов, разбиение текста на фрагменты (chunks) и сохранение их векторных представлений в базу данных.

2 Поиск фрагментов

Агент анализирует вопрос пользователя и находит наиболее релевантные куски текста в векторной базе.

3 Синтез ответа

LLM генерирует ответ, используя найденные фрагменты как контекст, и добавляет ссылки на источники.

4 Уточнение

Если информации недостаточно, агент может выполнить повторный поиск с другими ключевыми словами.

```
agent = Agent(..., knowledge=knowledge_base, search_knowledge=True)
```

Загрузка русскоязычной документации

knowledge_base.py

```python
from agno.knowledge.pdf import PDFKnowledgeBase, PDFReader
from agno.vectordb.qdrant import Qdrant

# 1. Настройка векторной базы данных
vector_db = Qdrant(
    collection="ru-docs",
    url="http://localhost:6333"
)

# 2. Создание базы знаний из PDF
knowledge = PDFKnowledgeBase(
    path="./docs/russian/",  # Путь к папке с документами
vector_db=vector_db,
    reader=PDFReader(chunk=True),
)

# 3. Загрузка данных (индексация)
if __name__ == "__main__":
    # recreate=False чтобы не пересоздавать индекс
    knowledge.load(recreate=False)

    # Async вариант:
# await knowledge.aload(recreate=False)
```

Векторная база данных

Используем **Qdrant** или **PgVector** для хранения векторных представлений (embeddings) текста. Это позволяет находить смысловые соответствия.

Чтение и чанкинг

`PDFReader(chunk=True)` автоматически разбивает длинные документы на небольшие фрагменты, чтобы они помещались в контекстное окно модели.

Стратегия загрузки

Используйте `recreate=False`, чтобы добавлять новые документы инкрементально, не удаляя старый индекс. Для production используйте асинхронный `aload()`.

Важно для русского языка

Убедитесь, что используемая модель эмбеддингов поддерживает кириллицу (например, `text-embedding-3-small` или `multilingual-e5`).

Создание агента-помощника

ru_docs_agent.py

```python
from agno.agent import Agent
from agno.db.sqlite import SqliteDb
from agno.models.anthropic import Claude
from agno.knowledge.pdf import PDFKnowledgeBase, PDFReader
from agno.vectordb.qdrant import Qdrant

# 1. Настройка векторной БД и базы знаний
vector_db = Qdrant(collection="ru-docs", url="http://localhost:6333")
knowledge = PDFKnowledgeBase(
    path="./docs/russian/",
    vector_db=vector_db,
    reader=PDFReader(chunk=True)
)
# knowledge.load(recreate=False) # Раскомментировать для загрузки
# 2. Создание агента
ru_docs_assistant = Agent(
    name="RU Docs Assistant",
    model=Claude(id="claude-sonnet-4-6"),
    db=SqliteDb(db_file="agno.db"),
    knowledge=knowledge,
    search_knowledge=True,
    add_history_to_context=True,
    num_history_runs=3,
    instructions=[
        "Отвечай на русском языке",
        "Для фактов используй базу знаний и добавляй ссылки",
        "Если контекст отсутствует — скажи об этом прямо"
    ],
    markdown=True,
)
```

knowledge & search_knowledge

Подключает базу знаний и разрешает агенту искать в ней информацию (RAG) при необходимости.

db=SqliteDb(...)

Персистентное хранилище сессий. Позволяет продолжать диалог после перезапуска.

add_history_to_context

Добавляет последние N сообщений в контекст для поддержания связности диалога.

instructions [...]

Специфичные инструкции для работы с документацией: язык ответа, правила цитирования и честность.

model=Claude(...)

Использование Claude 3.5 Sonnet, который отлично работает с русским языком и контекстом.

Запуск и тестирование

main.py

```python
if __name__ == "__main__":
    # 1. Простой тест одного запроса
    ru_docs_assistant.print_response(
        "Как настроить базу знаний из PDF?",
        stream=True
    )

    # 2. Интерактивный CLI режим
# Позволяет вести диалог в терминале
# ru_docs_assistant.cli_app(markdown=True)
# 3. Автоматическое тестирование
# response = ru_docs_assistant.run("...")
# assert "Qdrant" in response.content
```

RU Docs Assistant
Online

Как настроить базу знаний из PDF?

Searching knowledge base...

Для создания базы знаний из PDF документов в Agno используйте класс **PDFKnowledgeBase**. Вам также понадобится векторная база данных, например, **Qdrant**.

```
knowledge = PDFKnowledgeBase(
  path="./docs",
  vector_db=Qdrant(...)
)
```

Sources: agno_docs.pdf (p. 14)

Production deploy через AgentOS

app.py

```python
from agno.os import AgentOS
from fastapi import FastAPI
from agent import ru_docs_assistant

# Создаем экземпляр AgentOS с агентом и трассировкой
agent_os = AgentOS(
    agents=[ru_docs_assistant],
    tracing=True
)

# Получаем готовое FastAPI приложение
app: FastAPI = agent_os.get_app()
```

TERMINAL

```bash
$ export ANTHROPIC_API_KEY=***

$ uvx --python 3.12 \
  --with "agno[os]" \
  --with anthropic \
  fastapi dev app.py
```

ЧТО ВЫ ПОЛУЧАЕТЕ

Production REST API

Готовый FastAPI бэкенд с поддержкой streaming responses для мгновенного ответа пользователю.

Изоляция сессий

Stateless архитектура с разделением памяти по пользователям (user_id) и сессиям (session_id).

Auto Documentation

Автоматическая генерация Swagger UI / OpenAPI документации доступна по адресу /docs.

Native Tracing

Полная прозрачность работы агента: логирование вызовов инструментов, обращений к базе знаний и LLM.

Мониторинг через AgentOS UI

ПРОЦЕСС ПОДКЛЮЧЕНИЯ

1

Вход в систему

Перейдите на `os.agno.com` и авторизуйтесь в панели управления.

2

Создание OS

Нажмите кнопку **"Add new OS"** в меню и выберите тип **"Local"**.

3

Настройка URL

Введите адрес локального сервера: `http://localhost:8000`

4

Подключение

Задайте имя окружению (например, "Dev") и нажмите кнопку **"Connect"**.

ВОЗМОЖНОСТИ AGENTOS UI

- Чат с агентом
- Просмотр сессий
- Трассировка шагов
- Аудит инструментов
- Управление версиями

Расширенные возможности

Инструменты

Подключайте внешние API и инструменты через MCP протокол (DuckDuckGo, Google Search, Files и др.)

```
from agno.tools.mcp import MCPTools

agent = Agent(
    tools=[
        MCPTools(

url="https://docs.agno.com/mcp"
        ),
        DuckDuckGo()
    ],
    show_tool_calls=True
)
```

Источники знаний

Объединяйте разные типы данных (PDF, веб-сайты, CSV) в единую базу знаний с помощью CombinedKnowledgeBase.

```
from agno.knowledge import
CombinedKnowledgeBase

knowledge = CombinedKnowledgeBase(
    sources=[
        pdf_knowledge,
        website_knowledge,
        csv_knowledge
    ],
    vector_db=vector_db
)
```

Команды агентов

Создавайте команды специализированных агентов для решения сложных задач с разделением ответственности.

```
from agno.agent import Agent, AgentTeam

researcher = Agent(role="Researcher")
writer = Agent(role="Writer")

team = AgentTeam(
    agents=[researcher, writer],
    team_leader=True
)
team.print_response("Напиши пост")
```

Итоги и ресурсы

Что мы построили

Агент с Agentic RAG
Поддержка русской документации и контекстный поиск

Персистентная память
Хранение истории сессий в SQLite

Production API
FastAPI сервер через AgentOS

Мониторинг и трассировка
Полный контроль через AgentOS UI

Полезные ресурсы

- github.com/agno-agi/agno
- Quickstart Guide
- Knowledge & RAG Docs
- Cookbook Examples
- AgentOS UI (os.agno.com)